DEPARTMENT OF INFORMATION TECHNOLOGY

SMT. PARMESHWARIDEVI DURGADUTT TIBREWALA

LIONS JUHU COLLEGE

OF ARTS, COMMERCE AND SCIENCE

Affiliated to University of Mumbai

(Autonomous)

J.B. NAGAR, ANDHERI (E), MUMBAI-400059

Academic Year 2025-2026

COMPUTER VISION

For

Semester II

Submitted By:

Mr. Mohd Sahbaz Chaudhary

Msc.IT (Sem II)

SMT. PARMESHWARIDEVI DURGADUTT TIBREWALA

LIONS JUHU COLLEGE

OF ARTS, COMMERCE AND SCIENCE

Affiliated to University of Mumbai

(Autonomous)

J.B. NAGAR, ANDHERI (E), MUMBAI-400059

DEPARTMENT OF INFORMATION TECHNOLOGY

Certificate of Approval

This is to certify that practical entitled **"COMPUTER VISION"**, Undertaken at **SMT.PARMESHWARIDEVI DURGADUTT TIBREWALA LIONS JUHU COLLEGE OF ARTS, COMMERCE & SCIENCE.**
By **Mr. Mohd Sahbaz Chaudhary** **Seat No. MIT251001** in partial fulfilment of **M.Sc. (IT) master degree (Semester II)** Examination had not been submitted for any other examination and does not form of any other course undergone by the candidate. It is further certified that he has completed all required phases of the practical.

Internal Examiner **External Examiner**

HOD / In-Charge / Coordinator **Signature/**

Principal/Stamp

INDEX

Practical - 01

Aim: Perform geometric transformation using python

Theory: Geometric transformations

Geometric transformations change the position, size, and orientation of an image.
The main types are:

1. **Translation** – Moves image along x/y axis.
2. **Rotation** – Rotates image around a center point.
3. **Scaling** – Enlarges or shrinks the image.
4. **Shearing** – Tilts the shape of the image.
5. **Affine & Perspective** – Transforms using matrix operations for more complex warps.

Used in: Image editing, object detection, robotics vision, etc.

Steps to Add Packages in Python

1. Open terminal or command prompt.
2. Use pip command:
   - **pip install opencv-python matplotlib numpy**

How Many Packages Added?

3 packages added

- opencv-python
- matplotlib
- numpy

These are essential for image processing and plotting transformations.

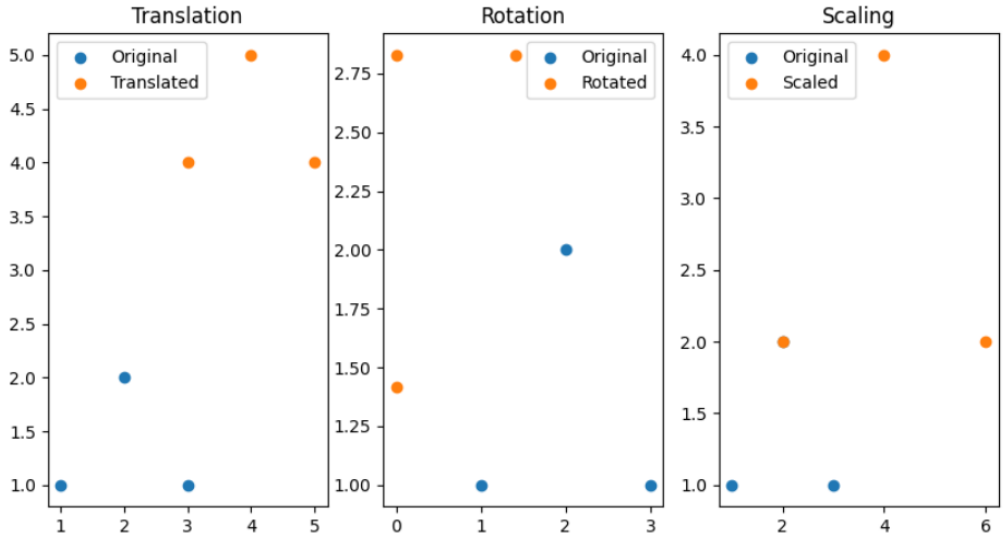
Code:

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.transforms as transforms

points=np.array([[1,1],[2,2],[3,1]])
translation_matrix=np.array([[1,0,2],[0,1,3],[0,0,1]])
translated_points=np.dot(translation_matrix,np.hstack([points,
np.ones((points.shape[0],1))]).T).T[:,:2]
theta=np.pi/4
```

```
rotation_matrix=np.array([[np.cos(theta),-
np.sin(theta),0],[np.sin(theta),np.cos(theta),0],[0,0,1]])
rotated_points=np.dot(rotation_matrix,np.hstack([points,
np.ones((points.shape[0],1))]).T).T[:,:2]
scaling_matrix=np.array([[2,0,0],[0,2,0],[0,0,1]])
scaled_points=np.dot(scaling_matrix,np.hstack([points,np.ones((points.shape[0],
1))]).T).T[:,:2]
plt.figure(figsize=(10,5))
plt.subplot(1,3,1)
plt.title('Translation')
plt.plot(points[:,0],points[:,1],'bo',label='original')
plt.plot(translated_points[:,0],translated_points[:,1],'r+',label='Translated')
plt.axis('equal')
plt.legend()
plt.subplot(1,3,2)
plt.title('Rotation')
plt.plot(points[:,0],points[:,1],'bo',label='original')
plt.plot(rotated_points[:,0],rotated_points[:,1],'r+',label='Rotated')
plt.axis('equal')
plt.legend()
plt.subplot(1,3,3)
plt.title('Scaling')
plt.plot(points[:,0],points[:,1],'bo',label='original')
plt.plot(scaled_points[:,0],scaled_points[:,1],'r+',label='Scaled')
plt.axis('equal')
plt.legend()
plt.show()
```

OUTPUT:

...



Practical - 02

Aim: Perform Image Stitching

THEORY:

Image stitching or photo stitching is the process of combining multiple photographic images with overlapping fields of view to produce a segmented panorama or high-resolution image. Commonly performed through the use of computer software, most approaches to image stitching require nearly exact overlaps between images and identical exposures to produce seamless results. It is also known as mosaicing. “Stitching” refers to the technique of using a computer to merge images together to create a large image, preferably without it being at all noticeable that the generated image has been created by computer. Algorithms for aligning images and stitching them into seamless photo-mosaics are among the oldest and most widely used in computer vision.

Code:

```
import cv2
import numpy as np

image1 = cv2.imread("C:\ Sahbaz\img.jpg")
image2 = cv2.imread("C:\ Sahbaz\img.jpg")
print("Image 1 shape:", image1.shape)
print("Image 2 shape:", image2.shape)

gray1 = cv2.cvtColor(image1, cv2.COLOR_BGR2GRAY)
gray2 = cv2.cvtColor(image2, cv2.COLOR_BGR2GRAY)

sift = cv2.SIFT_create()
keypoints1, descriptors1 = sift.detectAndCompute(gray1, None)
keypoints2, descriptors2 = sift.detectAndCompute(gray2, None)

matcher = cv2.BFMatcher()
matches = matcher.match(descriptors1, descriptors2)

matches = sorted(matches, key=lambda x: x.distance)

points1 = np.float32([keypoints1[match.queryIdx].pt for match in matches]).reshape(-1, 1, 2)
print("Number of points in points1:", len(points1))

points2 = np.float32([keypoints2[match.trainIdx].pt for match in matches]).reshape(-1, 1, 2)
print("Number of points in points2:", len(points2))
```

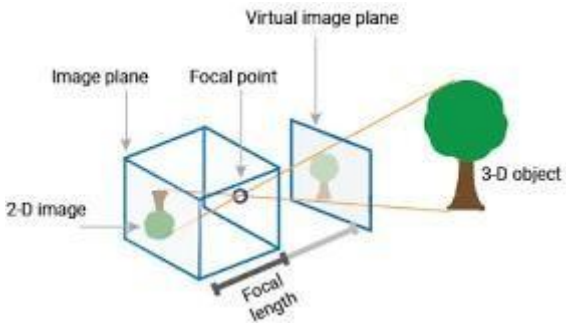
```
homography, _ = cv2.findHomography(points1, points2, cv2.RANSAC)

height, width = gray2.shape
stitched_image = cv2.warpPerspective(image1, homography, (width, height))

stitched_image[0:image2.shape[0], 0:image2.shape[1]] = image2

cv2.imshow('Stitched Image', stitched_image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output:

```
Image 1 shape: (200, 200, 3)
Image 2 shape: (200, 200, 3)
Number of points in points1: 87
Number of points in points2: 87
```

Practical - 03

Aim: Camera calibration



Camera calibration is a fundamental task in computer vision crucial in various applications such as 3D reconstruction, object tracking, augmented reality, and image analysis. Accurate calibration ensures precise measurements and reliable analysis by correcting distortions and estimating intrinsic and extrinsic camera parameters.
A camera is a device that converts the 3D world into a 2D image. A camera plays a very important role in capturing three-dimensional images and storing them in two-dimensional images. To know the mathematics behind it is extremely fascinating. The following equation can represent the camera.
x=PX

Here x denotes 2-D image point, P denotes camera matrix and X denotes 3-D world point.

$$\begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = \begin{bmatrix} p_1 & p_2 & p_3 & p_4 \\ p_5 & p_6 & p_7 & p_8 \\ p_9 & p_{10} & p_{11} & p_{12} \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

homogeneous image 3 x 1 | Camera matrix 3 x 4 | homogeneous world point 4 x 1

Figure1 vector representation of x=PX [1]
Camera calibration is frequently used word in image processing or computer vision field. The camera calibration method is intended to identify the geometric characteristics of the image creation process.

Types of Camera Calibration

1. Intrinsic or Internal Parameters

It allows mapping between pixel coordinates and camera coordinates in the image frame. E.g. optical center, focal length, and radial distortion coefficients of the lens.

2. Extrinsic or External Parameters

It describes the orientation and location of the camera. This refers to the rotation and translation of the camera with respect to some world coordinate system.

Note:

- Download some calibration images like check board pattern.
- Create one folder named calibration_images in the same directory as your python code.
- Copy that path and pass to code.

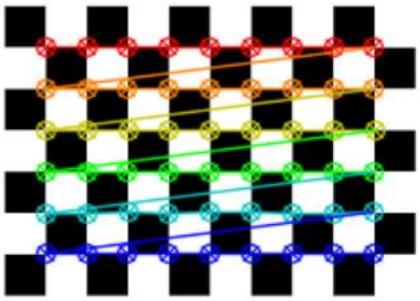
Code:

```
import numpy as np
import cv2
import glob
import os

# Define the number of corners in the chessboard
num_corners_x = 9
num_corners_y = 6
# Prepare object points, like (0,0,0), (1,0,0), ..., (8,5,0)
objp = np.zeros((num_corners_x * num_corners_y, 3), np.float32)
objp[:, :2] = np.mgrid[0:num_corners_x, 0:num_corners_y].T.reshape(-1, 2)
# Arrays to store object points and image points from all the images
objpoints = []  # 3d points
imgpoints = []  # 2d points
# Load images
images = glob.glob("C:\ Sahbaz\calib.png") img_shape =
None  # to store image size for calibration for fname in
images:
    img = cv2.imread(fname)
```

```
    if img is None:
        print(f"Could not read image {fname}")
        continue

    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    # Save image shape
    if img_shape is None:
        img_shape = gray.shape[::-1]

    # Find the chessboard corners
    ret, corners = cv2.findChessboardCorners(gray, (num_corners_x, num_corners_y), None)

    if ret:
        objpoints.append(objp)

        corners2 = cv2.cornerSubPix(
            gray, corners, (11, 11), (-1, -1),
            criteria=(cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 30,
0.001)
        )
        imgpoints.append(corners2)

        img = cv2.drawChessboardCorners(img, (num_corners_x, num_corners_y), corners2,
ret)
        cv2.imshow('img', img)
        cv2.waitKey(500)
    else:
        print(f"Chessboard not detected in image: {fname}")
cv2.destroyAllWindows()
```

```
# Check before calibrating
if len(objpoints) > 0 and len(imgpoints) > 0:
    ret, mtx, dist, rvecs, tvecs = cv2.calibrateCamera(objpoints, imgpoints, img_shape, None, None)
    # Save calibration results
    np.savez('calibration.npz', mtx=mtx, dist=dist, rvecs=rvecs, tvecs=tvecs)
    print("Camera matrix:")
    print(mtx)
    print("\nDistortion coefficients:")
    print(dist)
else:
    print("Not enough valid images for calibration. Please check your chessboard images.")
```

Output:



```
Camera matrix:
[[1.03955481e+03 0.00000000e+00 1.53881193e+02]
 [0.00000000e+00 1.04145168e+03 6.82307367e+01]
 [0.00000000e+00 0.00000000e+00 1.00000000e+00]]

Distortion coefficients:
[[ 1.18151099e+00 -9.65976028e+01  1.24794627e-03 -3.27170233e-03
   2.52382257e+03]]
```

Practical - 04

Aim: Face detection

THEORY

Detection techniques are used to identify objects like faces, eyes, or hands in an image or video.

Haar-Cascade Technique

- A machine learning-based object detection method.
- Uses **Haar features** (like edge, line, and rectangle patterns).
- Requires **training with lots of positive and negative images**.
- Uses **Cascade Classifier** – series of stages to quickly discard non-object areas.
- Fast and efficient, especially for **real-time face detection**.

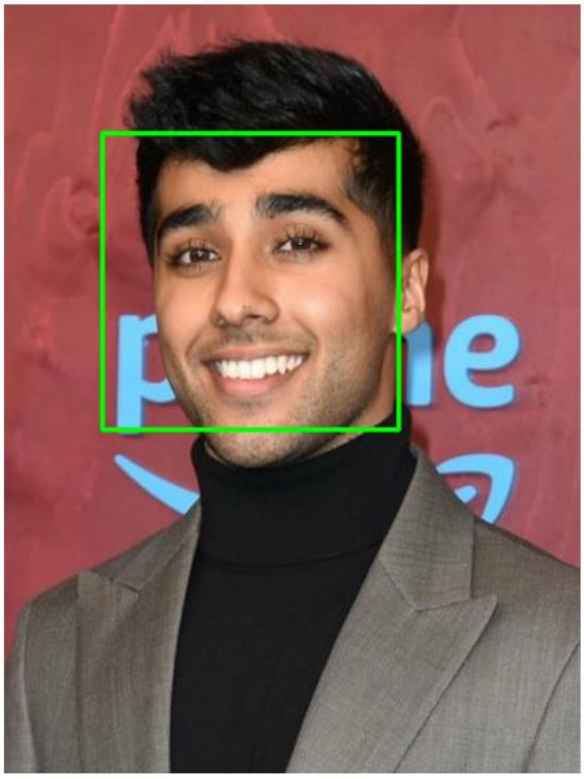
Commonly used in OpenCV (cv2.CascadeClassifier).

Code:

```
import cv2

# Load the pre-trained Haar Cascade face detector

face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades +
'haarcascade_frontalface_default.xml')

# Load the image

image = cv2.imread("C:\ Sahbaz\anw.jpg")

# Convert the image to grayscale (face detection works on grayscale images)

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Detect faces in the image

faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5, minSize=(30,
30))

# Draw rectangles around the faces

for (x, y, w, h) in faces:

    cv2.rectangle(image, (x, y), (x+w, y+h), (0, 255, 0), 2)
```

```
# Display the result
cv2.imshow('Face Detection', image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output:



Practical - 05

Aim: Object detection

THEORY :

Object Detection is the process of locating and identifying objects (like cars, people, faces) in images or videos.

It combines:

- **Classification** (what the object is)
- **Localization** (where the object is – using bounding boxes)

Common Techniques

1. **Haar Cascades** – Fast, traditional method for specific objects like faces.
2. **HOG + SVM** – Used for people detection (e.g., pedestrian detection).
3. **Deep Learning Methods**:
    - **YOLO (You Only Look Once)** – Fast, real-time detection.
    - **SSD (Single Shot Detector)** – Good balance of speed & accuracy.
    - **Faster R-CNN** – High accuracy, used for complex tasks.

Applications:

- Surveillance
- Autonomous vehicles
- Medical imaging
- Retail & logistics

➢ !pip install tensorflow

```
[1]
11s
!pip install tensorflow

Requirement already satisfied: tensorflow in /usr/local/lib/python3.12/dist-packages (2.19.0)
Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.4.0)
Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.6.3)
Requirement already satisfied: flatbuffers>=24.3.25 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (25.12.19)
Requirement already satisfied: gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.7.0)
Requirement already satisfied: google-pasta>=0.1.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.2.0)
Requirement already satisfied: libclang>=13.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (18.1.1)
Requirement already satisfied: opt-einsum>=2.3.2 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.4.0)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from tensorflow) (26.0)
Requirement already satisfied: protobuf!=4.21.0,!=4.21.1,!=4.21.2,!=4.21.3,!=4.21.4,!=4.21.5,<6.0.0dev,>=3.20.3 in /usr/local/lib/python3.12/dist-packages (f
Requirement already satisfied: requests<3,>=2.21.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.32.4)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from tensorflow) (75.2.0)
Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.17.0)
Requirement already satisfied: termcolor>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.3.0)
Requirement already satisfied: typing-extensions>=3.6.6 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (4.15.0)
Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.1.2)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.78.0)
Requirement already satisfied: tensorboard~=2.19.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.19.0)
Requirement already satisfied: keras>=3.5.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.13.2)
Requirement already satisfied: numpy<2.2.0,>=1.26.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.0.2)
Requirement already satisfied: h5py>=3.11.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.16.0)
Requirement already satisfied: ml-dtypes<1.0.0,>=0.5.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.5.4)
Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from astunparse>=1.6.0->tensorflow) (0.46.3)
Requirement already satisfied: rich in /usr/local/lib/python3.12/dist-packages (from keras>=3.5.0->tensorflow) (13.9.4)
Requirement already satisfied: namex in /usr/local/lib/python3.12/dist-packages (from keras>=3.5.0->tensorflow) (0.1.0)
Requirement already satisfied: optree in /usr/local/lib/python3.12/dist-packages (from keras>=3.5.0->tensorflow) (0.19.0)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow) (3.4.6)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow) (2.5.0)
```

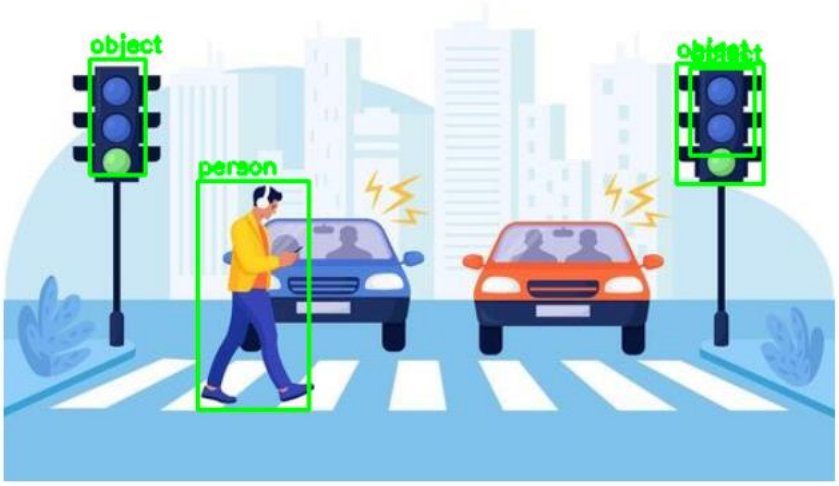
Code:

```
import numpy as np
import os
import tensorflow as tf
import cv2

# Load the pre-trained model
MODEL_NAME = 'ssd_mobilenet_v2_coco_2018_03_29'
PATH_TO_CKPT = os.path.join(MODEL_NAME, 'frozen_inference_graph.pb')
NUM_CLASSES = 90

detection_graph = tf.Graph()
with detection_graph.as_default():
    od_graph_def = tf.compat.v1.GraphDef()
    with tf.io.gfile.GFile(PATH_TO_CKPT, 'rb') as fid:
        serialized_graph = fid.read()
        od_graph_def.ParseFromString(serialized_graph)
        tf.import_graph_def(od_graph_def, name='')

# Load label map
#PATH_TO_LABELS = os.path.join('data', 'mscoco_label_map.pbtxt

PATH_TO_LABELS = "C:\ Sahbaz\mscoco_label_map.pbtxt" category_index =
{}
with open(PATH_TO_LABELS, 'r') as f:
    lines = f.readlines()
    for line in lines:
        if 'id:' in line:
            id_index = int(line.strip().split(':')[1])
        if 'display_name:' in line:
            name = line.strip().split(':')[1].strip().strip('"')
            category_index[id_index] = {'name': name}

# Function to perform object detection
def detect_objects(image):
    with detection_graph.as_default():
        with tf.compat.v1.Session(graph=detection_graph) as sess:
            # Expand dimensions since the model expects images to have shape: [1, None, None,
3]
            image_expanded = np.expand_dims(image, axis=0)
            image_tensor = detection_graph.get_tensor_by_name('image_tensor:0')
            # Each box represents a part of the image where a particular object was detected.
            boxes = detection_graph.get_tensor_by_name('detection_boxes:0')
            # Each score represents the level of confidence for each of the objects.
```

```
        # The score is shown on the result image, together with the class label.
        scores = detection_graph.get_tensor_by_name('detection_scores:0')
        classes = detection_graph.get_tensor_by_name('detection_classes:0')
        num_detections = detection_graph.get_tensor_by_name('num_detections:0')
        # Actual detection.
        (boxes, scores, classes, num_detections) = sess.run([boxes, scores, classes,
num_detections],feed_dict={image_tensor: image_expanded})
        # Visualization of the results of a detection.
        for i in range(len(scores[0])):
            if scores[0][i] > 0.5:  # Adjust confidence threshold as needed
                class_id = int(classes[0][i])
                class_name = category_index[class_id]['name']
                score = float(scores[0][i])
                ymin, xmin, ymax, xmax = boxes[0][i]
                (left, right, top, bottom) = (xmin * image.shape[1], xmax * image.shape[1], ymin
* image.shape[0], ymax * image.shape[0])
                cv2.rectangle(image, (int(left), int(top)), (int(right), int(bottom)), (0, 255, 0), 2)
                cv2.putText(image, '{}: {:.2f}'.format(class_name, score), (int(left), int(top - 5)),
cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)
    return image

# Object detection on an image

input_image = cv2.imread("C:\ Sahbaz\ob. jpg")
output_image = detect_objects(input_image)
cv2.imshow('Object Detection', output_image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

OUTPUT:

```
Detected objects:
person (0.84)
object (0.67)
object (0.53)
object (0.52)
```

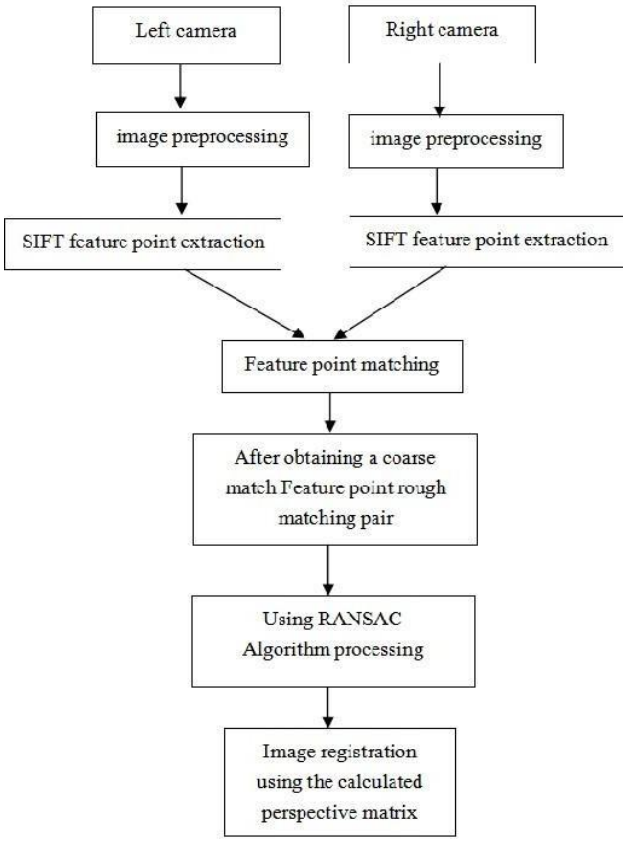


Practical - 06

Aim: Pedestrian detection

THEORY:

Pedestrian detection is an advanced driver assistance system (ADAS). It uses sensors, cameras, and artificial intelligence to identify pedestrians near vehicles. By analyzing the surrounding environment, the system can recognize human forms and movements, helping to prevent or mitigate potential collisions between vehicles and pedestrians.



How Does Pedestrian Detection Work?

Pedestrian detection systems combine various technologies to effectively detect and track pedestrians. These technologies work in tandem to create a comprehensive safety net around the vehicle.

High-resolution cameras play a pivotal role in pedestrian detection. Strategically positioned around the vehicle, these cameras capture visual information about the surrounding area, including sidewalks, crosswalks, and pedestrians. The system can identify human forms and movements by analyzing the captured images in real time, distinguishing pedestrians from other objects.

Radar sensors are another crucial component of pedestrian detection systems. These sensors use radio waves to detect objects within a specific range of the vehicle. With their ability to accurately measure distance and speed, radar sensors can detect pedestrians, even in challenging conditions such as poor visibility or adverse weather.

Incorporating Lidar technology takes pedestrian detection to the next level. Lidar systems employ laser beams to create a detailed 3D map of the environment surrounding the vehicle. This enables precise detection and tracking of pedestrians, regardless of lighting conditions. The system can effectively differentiate pedestrians from their surroundings by constructing a comprehensive spatial representation.

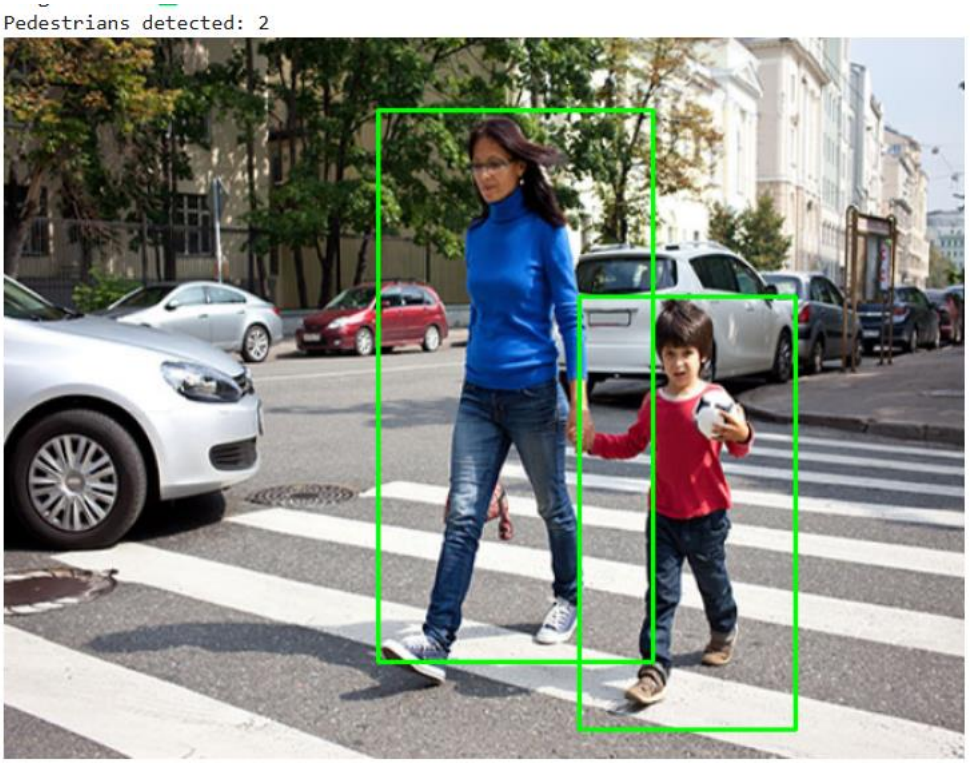
However, the power of pedestrian detection lies in hardware, advanced algorithms, and artificial intelligence. These intelligent systems analyze the data collected from cameras, radar, and Lidar. By processing this information, they can accurately identify pedestrians, distinguishing them from other objects on the road. Additionally, the system can predict pedestrian behavior, anticipate potential hazards, and provide timely warnings to the driver. This multi-sensor approach ensures high accuracy, enabling the system to react promptly to potential collision risks.

Code:

```
import cv2

# Load the pre-trained pedestrian detector

pedestrian_cascade = cv2.CascadeClassifier(cv2.data.haarcascades +
'haarcascade_fullbody.xml')


# Load the input image ** walking ppl image

image = cv2.imread("C:\ Sahbaz\ped.jpg")


# Convert the image to grayscale

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)


# Detect pedestrians in the image

pedestrians = pedestrian_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=1,
minSize=(5, 5))


# Draw rectangles around the detected pedestrians

for (x, y, w, h) in pedestrians:
```

```
    cv2.rectangle(image, (x, y), (x+w, y+h), (0, 255, 0), 2)

# Display the image with pedestrian detections
cv2.imshow('Pedestrian Detection', image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output:



Practical - 07

Aim: Feature extraction using RANSAC

THEORY:

What is RANSAC?

RANSAC (Random Sample Consensus) is an iterative algorithm used to estimate parameters of a mathematical model from a set of observed data that contains **outliers**. It's especially useful in computer vision, robotics, and data fitting where noisy data is common.

How does RANSAC work (practical steps)?

1. **Randomly select a small subset** of data points.
2. **Fit a model** to this subset.
3. **Test all other data points** against this model.
4. Count how many points (called **inliers**) fit the model within a predefined tolerance.
5. Repeat steps 1–4 for a number of iterations.
6. **Choose the model with the highest number of inliers** as the final model.

Why use RANSAC?

- It's **robust to outliers**.
- Can find a reliable model even when a large portion of the data is noisy.
- Simple and effective for problems like **line fitting**, **homography estimation**, or **3D reconstruction**.

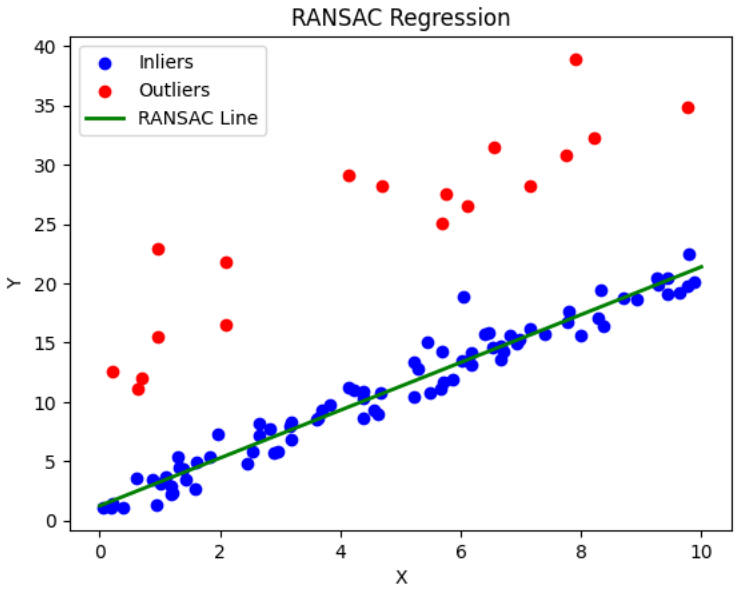
Code:

```
import numpy as np
from sklearn.linear_model import RANSACRegressor
import matplotlib.pyplot as plt

# Generate some noisy data points
np.random.seed(0)
x = np.random.uniform(0, 10, 100)
y = 2 * x + 1 + np.random.normal(0, 1, 100)

# Add outliers
outliers_index = np.random.choice(100, 20, replace=False)
y[outliers_index] += 10 * np.random.normal(0, 1, 20)

# Stack the points for RANSAC
data = np.vstack((x, y)).T

# Initialize RANSAC model (using sklearn)
ransac = RANSACRegressor()
```

```
# Fit the model
ransac.fit(data[:, 0].reshape(-1, 1), data[:, 1])

# Extract the inliers and outliers
inlier_mask = ransac.inlier_mask_
outlier_mask = np.logical_not(inlier_mask)

# Extract the line parameters
line_slope = ransac.estimator_.coef_[0]
line_intercept = ransac.estimator_.intercept_

# Plot the data points
plt.scatter(data[inlier_mask][:, 0], data[inlier_mask][:, 1], c='b', label='Inliers')
plt.scatter(data[outlier_mask][:, 0], data[outlier_mask][:, 1], c='r', label='Outliers')

# Plot the fitted line
plt.plot(x, line_slope * x + line_intercept, color='g', label='RANSAC line')

plt.xlabel('X')
plt.ylabel('Y')
plt.legend()
plt.grid(True)
plt.show()
```

OUTPUT:

```
Inliers: 82
Outliers: 18
```



Practical - 08

Aim: Colorization of an image

THEORY:

What is Colorization?

Colorization is the process of adding color to grayscale or black-and-white images. It's widely used in restoring old photographs, film colorization, and in AI-powered image enhancement.

Colorization Techniques:

1. **Manual Colorization**
   - Artists manually paint colors over grayscale images using graphic software.
   - Time-consuming but highly controlled and accurate.
2. **Example-Based Colorization**
   - Transfers colors from a reference color image to a grayscale image based on similarity in texture, shape, or structure.
   - Semi-automatic method.
3. **Learning-Based (Deep Learning) Colorization**
   - Uses **Convolutional Neural Networks (CNNs)** trained on large datasets of color images.
   - Automatically predicts suitable colors for grayscale images.
   - Example: **DeOldify**, **Let there be Color!**
4. **Scribble-Based Colorization**
   - User adds colored scribbles or hints on the grayscale image.
   - Algorithm propagates these colors to surrounding areas based on intensity and edges.

Why use Colorization?

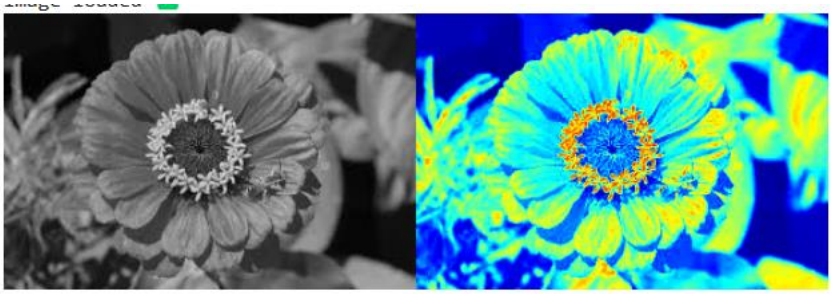
- Restoring historical photos and films.
- Enhancing visual appeal.
- Preprocessing for other computer vision tasks.
- Aesthetic or artistic applications.

Code:

```
import cv2
import numpy as np
# Load the grayscale image
gray_image = cv2.imread("C:\ Sahbaz\flower.jpg", cv2.IMREAD_GRAYSCALE)
# Convert grayscale image to BGR (3-channel) for colorization
color_image = cv2.cvtColor(gray_image, cv2.COLOR_GRAY2BGR)
# Define a basic colorization lookup table
```

```
color_lookup_table = np.zeros((256, 1, 3), dtype=np.uint8)
for i in range(256):
    color_lookup_table[i, 0, 0] = i  # Blue channel
    color_lookup_table[i, 0, 1] = 127  # Green channel
    color_lookup_table[i, 0, 2] = 255 - i  # Red channel
# Apply the colorization lookup table to the grayscale image
colorized_image = cv2.LUT(color_image, color_lookup_table)
# Display the original grayscale image and the colorized image
cv2.imshow('Grayscale Image', gray_image)
cv2.imshow('Colorized Image', colorized_image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output



Practical - 09

Aim: Image matting and composting

THEORY:

Image Matting is the process of accurately estimating the foreground object in images and videos. It is a very important technique in image and video editing applications, particularly in film production for creating visual effects. In case of image segmentation, we segment the image into foreground and background by labeling the pixels. Image segmentation generates a binary image, in which a pixel either belongs to foreground or background. However, Image Matting is different from the image segmentation, wherein some pixels may belong to foreground as well as background, such pixels are called partial or mixed pixels. In order to fully separate the foreground from the background in an image, accurate estimation of the alpha values for partial or mixed pixels is necessary.

Code:

```
import cv2

import numpy as np


def estimate_alpha(image, trimap):

    # Convert to float

    image = image.astype(np.float32) / 255.0


    # Normalize trimap to [0, 1]

    trimap = trimap.astype(np.float32) / 255.0


    # Compute alpha matte using Closed-Form matting

    foreground = np.where(trimap > 0.95, 1.0, 0.0)  # Foreground mask

    alpha = np.where(trimap > 0.05, 1.0, 0.0)  # Alpha initialization

    for _ in range(5):  # Iterative refinement

        alpha = (image[:, :, 0] - image[:, :, 2] * alpha) / (1e-12 + foreground + (1.0 - trimap) *
alpha)

        alpha = np.clip(alpha, 0, 1)


    return alpha
```

```
# Example usage
if __name___== "__main__":
    # Read image and trimap
    image = cv2.imread("C:\ Sahbaz\owl.jpg")
    trimap = cv2.imread("C:\ Sahbaz\bg.jpg", cv2.IMREAD_GRAYSCALE)

    # Estimate alpha matte
    alpha = estimate_alpha(image, trimap)

    # Save or display alpha matte
    cv2.imshow("Alpha Matte", alpha)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
```

Output: